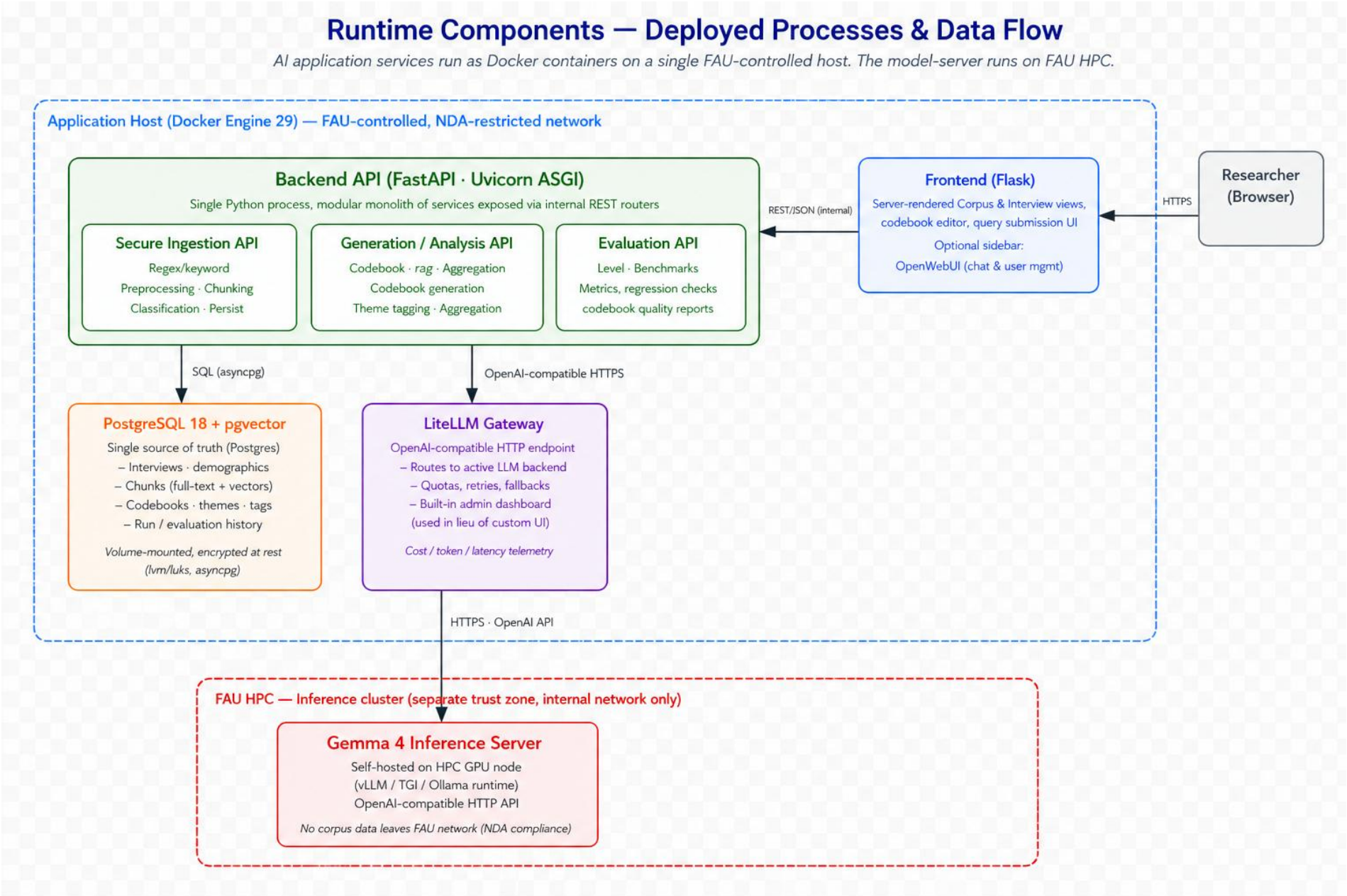
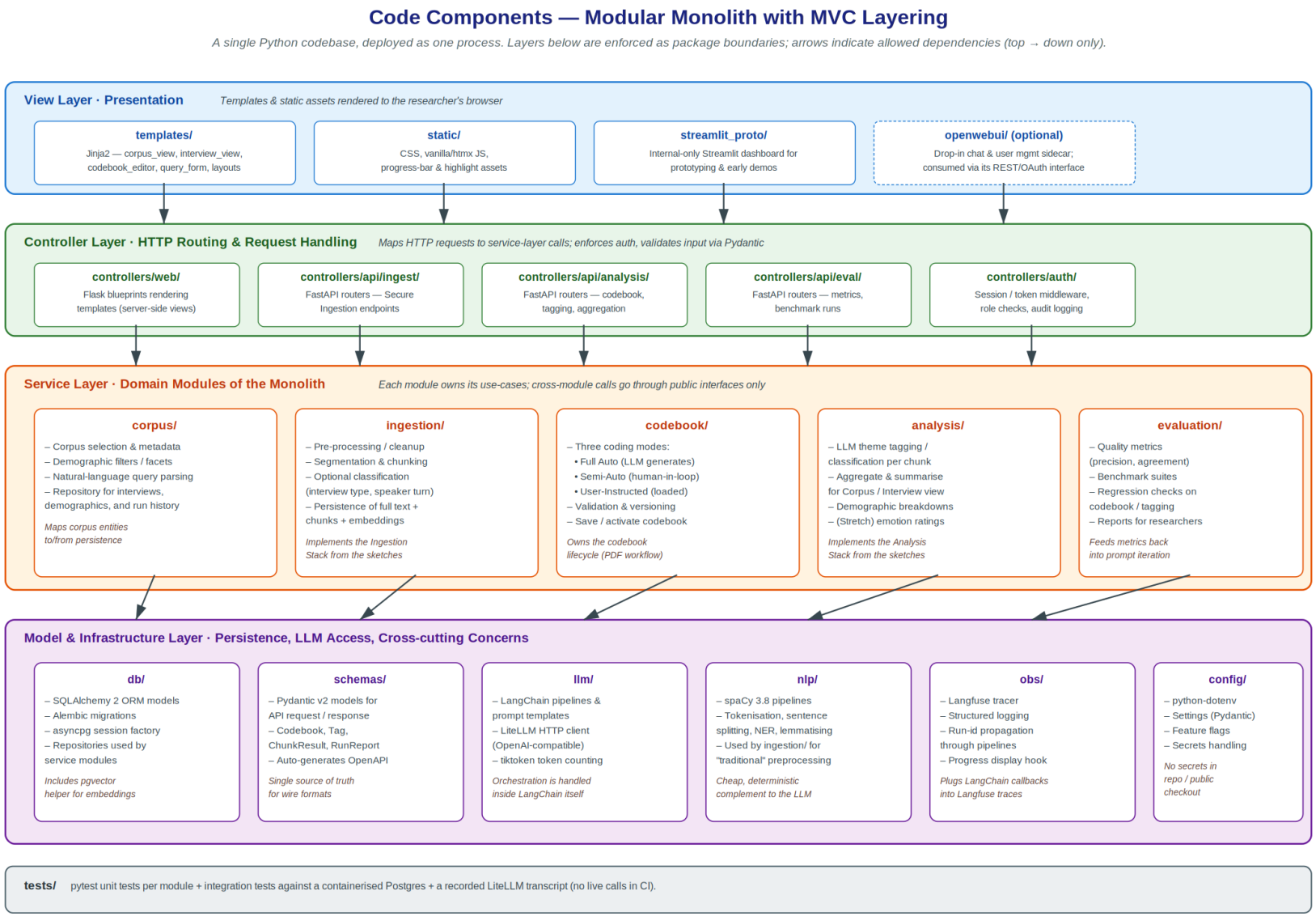
1. Runtime Components

Processes that exist at runtime, the trust boundaries they sit inside, and the data flows between them.



2. Code Components

Internal package structure of the monolith, organised into MVC layers plus an explicit service layer that owns the domain logic.



3. Technology Stack

All components are open-source. No corpus content is sent to third-party hosted services. Versions reflect the latest stable releases targeted at project kick-off and will be pinned via a lockfile.

Layer / Component	Technology	Version	Role in the system
Runtime & Deployment			
Language	Python	3.14	Backend implementation language for all services.
Container	Docker Engine	29	Packages all services for reproducible deployment on the host.
ASGI server	Uvicorn	0.46	Hosts the FastAPI backend; chosen for async LLM I/O.
Config	python-dotenv	1.2	Loads environment variables; keeps secrets out of source.
Backend API & Validation			
API framework	FastAPI	0.136	REST surface for ingestion, analysis, and evaluation endpoints.
Validation	Pydantic	2.13	Request / response schemas; auto-generated OpenAPI docs.
HTTP client	HTTPX	0.28	Async outbound calls to LiteLLM and Langfuse.
Tokenization	tiktoken	0.12	Token counts for prompt budgeting and chunk sizing.
Frontend			
Web framework	Flask + Jinja2	latest	Server-rendered Corpus / Interview views and codebook editor.
Chat / users	OpenWebUI (opt.)	latest	Optional sidecar for chat-style interaction & user management.
LLM Stack			
Workflow lib	LangChain	1.2	In-process orchestration of LLM steps and tools.
LLM gateway	LiteLLM	1.83	OpenAI-compatible router; admin dashboard for ops.
Traditional NLP	spaCy	3.8	Tokenisation, sentence splitting, NER for ingestion.
Persistence			
Database	PostgreSQL	18	Single source of truth for all application data.
Vector ext.	pgvector	0.8	Embeddings stored alongside relational data
ORM / SQL	SQLAlchemy	2	Domain models, repositories, and migrations.
PG driver	asyncpg	0.31	High-performance async PostgreSQL connectivity.
Quality			
Testing	pytest	9	Unit and integration tests; CI gate before merge.

4. Explanation of the Diagrams and Choices

This section walks through the two diagrams, explains why each major choice was made, and records the trade-offs the team accepted.

Architectural style: Modular Monolith with MVC

The system is deployed as a single Python process rather than a constellation of microservices. Inside that process, code is partitioned along two complementary axes. The MVC axis separates Views/Frontend, Controllers and Models. The service axis groups domain logic into modules (corpus, ingestion, codebook, analysis, evaluation) each with a public interface and private internals. Dependencies flow strictly downward in Figure 2: a controller may call a service, a service may use the infrastructure layer, but never the other way around. This convention lets the monolith be split into separate services later.

Why this style: Local-first development, one Docker compose file for the whole system, and one Postgres database to back up are all direct consequences. The MVC discipline keeps the code understandable for new contributors, and the service modules give us seams for refactoring without forcing premature distribution.

Walkthrough of the runtime diagram

A researcher's browser reaches the Flask frontend over HTTPS. The frontend renders the Corpus and Interview views and forwards the user's NL queries and codebook edits to the FastAPI backend via internal REST/JSON. The backend exposes three logical APIs that correspond directly to the planning sketches: Secure Ingestion (the Ingestion Stack), Generation / Analysis (the Analysis Stack), and Evaluation. The backend persists everything to a single PostgreSQL database with the pgvector extension where embeddings live next to the relational data, eliminating the need for a separate vector store. LLM calls go out through LiteLLM, which presents an OpenAI-compatible interface and routes to one or more Gemma inference servers running on FAU HPC. The HPC zone is shown as a separate trust boundary to make the data-flow explicit: text leaves the application host only to reach a model that is itself inside the FAU network.

Walkthrough of the code diagram

The View layer is intentionally thin: server-side templates, static assets and an optional OpenWebUI interface that would replace the team's hand-built chat and user-management surfaces with a maintained one. The Controller layer splits along the same lines as the runtime APIs and is the only place HTTP concerns (auth, request shape, status codes) live. The Service layer carries the domain. “ingestion/” implements the Ingestion Stack from the sketch: preprocessing with spaCy, segmentation, optional classification (kept behind a feature flag while we evaluate its value), and persistence. “codebook/” owns the three coding modes shown in the workflow PDF (Full-Auto, Semi-Auto, User-Instructed) and the codebook lifecycle. “analysis/” performs theme tagging on chunks, aggregates results into Corpus and Interview views, and provides the demographic breakdowns the project description calls out; the optional emotion-rating dimension lives here as well. “evaluation/” computes quality metrics and feeds them back into prompt iteration. The Model & Infrastructure layer hides every cross-cutting concern. SQLAlchemy + asyncpg + Alembic for persistence, Pydantic schemas as the single source of truth for wire formats, a thin LiteLLM client wrapping LangChain pipelines, spaCy utilities for traditional NLP and dotenv-based configuration.

Key choices

Orchestration is delegated to LangChain. The Analysis Stack sketch contained an Orchestrator block marked “Skip”. We adopt that decision: LangChain already provides chains, agents, retries, and callbacks. If the project later requires scheduled or long-running jobs across multiple workers, this decision will be revisited.

LiteLLM provides both gateway and dashboard. The same sketch noted “LiteLLM” next to a Dashboard block. We use LiteLLM's built-in admin UI for routing, quotas, and basic ops visibility, and rely on Langfuse for the deeper LLM-trace analytics. Building a custom ops dashboard is out of scope.

PostgreSQL with pgvector serves as a dedicated vector database with full metadata control. Embeddings are first-class data and they are always queried alongside relational filters (demographics, codebook version, run id). Co-locating them in Postgres avoids a second system to operate, secure, and back up.

Gemma on FAU HPC, accessed via OpenAI-compatible API. Self-hosting an open model on FAU HPC was a hard constraint of the project. LiteLLM gives us a stable, swappable interface so a second model (e.g. for benchmarking, LLM-as-judge evaluation, or a fallback) can be added without touching service-layer code.

Frontend remains pragmatic. The team will write the production frontend in Flask with server-rendered templates. Where scope allows, OpenWebUI will be adopted as a drop-in chat and user-management layer to reduce both implementation effort and the security surface. Building authentication ourselves under an NDA is the kind of risk we should not take if a vetted, maintained alternative covers the requirement.